



← Back to student guides

✦ Airflow

Orchestration

3 levels

113 sections



The Complete Guide

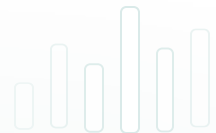
Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.

DAG

SCHEDULER

EXECUTOR

WORKER



17

sections

27

examples



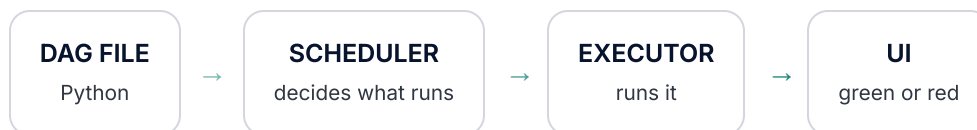


sensibly, connect to a database, backfill history, and read the interface well enough to debug a red run. Mid-level and Senior take the same topics further; nothing here is thrown away.

Each section ends with a **Try it** task. Do them as you go. They take a few minutes each, and these concepts only stick once you have watched your own DAG turn green, then deliberately red.

What Airflow is, and the problem it solves

Airflow is a platform for **writing, scheduling, and monitoring workflows in Python**. You define a workflow as code, Airflow works out the order tasks must run in, executes them on a schedule, retries the ones that fail, and gives you a web interface showing what happened.



The diagram is the whole idea. Compare it to what came before.

Before this, a data pipeline was a chain of cron jobs. `0 2 * * * /opt/etl/extract.sh` at 2am, then `0 3 * * * /opt/etl/transform.sh` at 3am, and a hope that extract finished within the hour. When it did not, transform ran on yesterday's data and nobody noticed until a dashboard looked wrong on Thursday. There was no dependency graph, no retry, no history, and no answer to "did last Tuesday's run succeed?" other than grepping logs on whichever box happened to have them.

Airflow replaces the hope with a dependency graph. Transform does not run at 3am; it runs when extract has succeeded. If extract fails, transform does not run at all, and you get told.

Two consequences of the design explain most of what follows. Notice them now rather than discovering them later.

Workflows are Python files. That means your pipeline is versioned, reviewed, and diffed like any other code, and it also means Airflow `executes` your file to discover the DAG, which has



deliberately bad at processing data itself. A task that loads a 40 GB dataframe into the worker's memory is a misuse; a task that tells Spark or BigQuery or dbt to do that work is correct. This distinction separates people who have operated Airflow from people who have only read about it.

What people use it for:



ETL and ELT

Extract from sources, load into a warehouse, transform. All on a schedule, in dependency order, with retries.



ML pipelines

Feature builds, scheduled retraining, batch inference, and the checks that gate a model release.



Reporting

Nightly aggregations that must complete before people arrive, with alerting when they do not.



Operational chores

Backups, cleanups, data-quality checks, and anything currently living in a forgotten crontab.

You need little to follow along: Python, Docker, and a terminal. The official Docker Compose setup gets you a working Airflow in about five minutes, and a DAG with three `EmptyOperator` tasks is enough to learn the whole scheduling model.

► TRY IT

1. Find a scheduled job you or your team depend on: a cron entry, a Jenkins job, a scheduled notebook.
2. Write down what happens if it fails at 2am. Who finds out, and how long does it take?
3. Write down what happens to the job that runs after it.

Expected result – usually "nobody finds out until someone notices a wrong number", and "the next job runs anyway on stale data". Those two answers are the



The architecture: five components, two planes

You will hit error messages that only make sense if you know which component is complaining, so learn the shape now.

Component	Job	You notice it when
Scheduler	Parses DAG files, decides which task instances are ready, queues them	Nothing runs, or runs late
Executor	Hands queued tasks to workers	Tasks sit in <code>queued</code> forever
Worker	Actually runs your task code	Your task fails, or runs out of memory
Metadata database	Every DAG run, task instance, connection, variable	The UI is slow, or state looks wrong
Webserver	The UI and REST API	You cannot see anything, but pipelines still run

WHAT YOU WRITE

`dags/` : plain Python files

Re-parsed every ~30s. The scheduler `executes` each file to find DAG objects



CONTROL PLANE

Scheduler



Webserver

A window onto the database. Stop it and pipelines keep running



EXECUTION PLANE

Executor

Hands queued tasks to workers. Tasks stuck in **queued** point here

Worker

Runs your task code. Failures and OOM kills happen here

Triggerer

Deferred tasks. Ignore it until Mid level

Two facts to carry forward: the scheduler executes your DAG files continuously, so slow top-level code slows the entire scheduler, and the database, not the file, records what happened. When the UI shows a task your file no longer defines, both are correct; they answer different questions.

Two things about this picture matter immediately.

The scheduler re-reads your DAG files continuously. By default every thirty seconds it scans the folder for changes, and it **executes** each file to find DAG objects. Slow top-level code therefore slows down the whole scheduler, which is the single most common cause of "why is Airflow so sluggish", and the reason for a rule we will state properly in a moment.

The metadata database is the source of truth, not your DAG file. The file says what **should** exist; the database records what **did** happen. When the UI shows a task as failed and your file no longer contains that task, both are correct. They are answering different questions.

There is a sixth component you will meet at Mid level, the **triggerer**, which handles deferred tasks. Ignore it for now.



1. Get Airtlow running: download the official `docker-compose.yaml` and run `docker compose up -d`.
2. Run `docker compose ps` and match each container to a row in the table above.
3. Open `localhost:8080` and log in. The scheduler is a separate container from the webserver.
4. Stop just the webserver: `docker compose stop airflow-webserver`. Wait a minute, then start it again and check whether any scheduled runs were missed.

Expected result – the UI disappears but the scheduler keeps working. Runs continue while you cannot see them. Prove that to yourself once and the webserver stops looking like the engine.

Your first DAG, line by line

Enough theory. Create a file called `hello_dag.py` in your `dags/` folder.

dags/hello_dag.py

Copy

```
from datetime import datetime
from airflow import DAG
from airflow.operators.empty import EmptyOperator
from airflow.operators.bash import BashOperator

with DAG(
    dag_id="hello_world",
    start_date=datetime(2024, 1, 1),
    schedule="@daily",
    catchup=False,
    tags=["tutorial"],
) as dag:

    start = EmptyOperator(task_id="start")

    say_hello = BashOperator(
        task_id="say_hello",
        bash_command="echo 'Hello from Airflow'",
    )
```



```
        bash_command="date",  
    )  
  
    start >> say_hello >> show_date
```

Wait up to thirty seconds and it appears in the UI. Unpause it with the toggle and watch it run.

Now let me walk through every line, because this small file contains the entire conceptual model.

`with DAG(...) as dag:` creates the DAG object. Everything defined inside the block is attached to it automatically, which is why you rarely see `dag=dag` on each task in modern code.

`dag_id="hello_world"` is the unique identifier. It appears in the UI, in the CLI, and in the database, so renaming it creates a `new` DAG and abandons the old one's history.

`start_date=datetime(2024, 1, 1)` is the date from which Airflow considers intervals to exist, not the day you wrote the file. It is the single most misunderstood parameter in Airflow and it gets its own section shortly.

`schedule="@daily"` is how often it runs. A cron string, a preset like this, or `None` for manual-only.

`catchup=False` tells Airflow **not** to run every interval between `start_date` and now. Leave this out on a DAG with a 2022 start date and you will trigger seven hundred runs the moment you unpause it. More on this shortly too.

`tags=["tutorial"]` groups DAGs in the UI. Free, and worth using from the first DAG.

`EmptyOperator` does nothing. It exists as a marker, a clean entry or exit point in a graph, and it is useful.

`BashOperator` runs a shell command on the worker.

`start >> say_hello >> show_date` sets the dependencies. That `>>` is Airflow's bitshift operator, overloaded to mean "then".



depend on itself. Airflow enforces this: a cycle is an import error, not a runtime surprise. That guarantee is what lets the scheduler always compute an execution order.

► TRY IT

1. Create the DAG above, wait for it to appear, unpause it, and watch the graph turn green.
2. Click into `say_hello` and read the log. Find your echoed line.
3. Add a fourth task and put it in parallel: `start >> [say_hello, show_date]` .
4. Now create a cycle on purpose, `show_date >> say_hello` as well, and read the error Airflow gives you.

Expected result – a green run, a readable log, and a parallel branch. The deliberate cycle produces an import error in the UI rather than a broken run, which is the acyclic guarantee doing its job.

Tasks, operators, and the dependency graph

Three words that get used loosely and mean different things.

Term	Is
Operator	A class describing a kind of work : <code>BashOperator</code> , <code>PythonOperator</code>
Task	An instance of an operator inside a DAG, with a <code>task_id</code>
Task instance	One specific run of that task for one specific interval

So `BashOperator` is the operator, `say_hello` is the task, and "`say_hello` for 2024-05-01" is the task instance. The UI shows task instances; your file defines tasks.

The operators you will use early:



<code>EmptyOperator</code>	Nothing, a graph marker
<code>BashOperator</code>	A shell command
<code>PythonOperator</code>	A Python callable
<code>SQLExecuteQueryOperator</code>	SQL against a connection
<code>DockerOperator</code> / <code>KubernetesPodOperator</code>	A container
Provider operators	S3, BigQuery, Postgres, Slack, dbt, and hundreds more

Dependencies have four shapes worth knowing:

PYTHON

Copy

```
a >> b                # a, then b
a << b                # b, then a (the same thing, read backwards)

a >> [b, c] >> d       # fan out to b and c in parallel, then fan in to d

chain(a, b, c, d)      # a >> b >> c >> d, without the arrow soup
cross_downstream([a, b], [c, d]) # every one of a,b feeds every one of c,d
```

PYTHON

Copy

```
from airflow.models.baseoperator import chain, cross_downstream
```

Prefer `chain()` once you have more than three tasks

`a >> b >> [c, d] >> e >> [f, g] >> h` is technically fine and unreadable in review.
`chain()` and `cross_downstream()` express the same graph in a form a colleague can scan.

► TRY IT



3. Rewrite the same graph using `chain()` and confirm the Graph view is identical.

4. Make one of the parallel tasks fail with `bash_command="exit 1"` and watch what happens to the task after it.

Expected result – a diamond in the Graph view, and a downstream task marked `upstream_failed` rather than `run`. That state is Airflow's core promise: a failure upstream stops the work that depended on it.

The TaskFlow API: DAGs that read like Python

Everything above is the classic operator style, and you will read a lot of it. For Python work, modern Airflow has a cleaner form.

dags/taskflow_example.py

Copy

```
from datetime import datetime
from airflow.decorators import dag, task

@dag(
    dag_id="taskflow_example",
    start_date=datetime(2024, 1, 1),
    schedule="@daily",
    catchup=False,
    tags=["tutorial"],
)
def etl_pipeline():

    @task
    def extract():
        return {"records": [1, 2, 3, 4, 5]}

    @task
    def transform(data: dict):
        return {"total": sum(data["records"])}

    @task
```



```
load(transform(extract()))  
  
etl_pipeline()
```

Read the last line inside the function: `load(transform(extract()))` . That single expression does two things at once. It declares the dependency chain **and** passes the return values along it. No `>>` , no manual data passing.

✓ TaskFlow: for Python work

- Dependencies inferred from function calls
- Return values passed automatically
- Type hints and normal Python signatures
- Much less boilerplate

✗ Classic operators: for everything else

- Explicit `>>` dependencies
- Data passed manually through XCom
- The only option for non-Python operators
- What most existing DAGs look like

They mix freely in one DAG, which is what you will do:

PYTHON

Copy

```
raw = extract()                                # TaskFlow task  
notify = BashOperator(task_id="notify", bash_command="echo done")  
raw >> notify                                  # classic dependency
```

Use TaskFlow for Python, operators for systems

If the work is Python you wrote, TaskFlow is shorter and clearer. If the work is "run this SQL", "start this Spark job", "wait for this file", use the provider operator built for it.

Reimplementing an operator inside a `@task` is a common beginner instinct and almost always the wrong trade.

► TRY IT

1. Write the TaskFlow DAG above and confirm the Graph view shows `extract` → `transform` → `load` without any `>>` .
2. Read the `transform` log and find where the value from `extract` arrived.



Expected result – a dependency graph inferred from function calls, and a mixed DAG where both styles coexist. Step four shows the flip side: a task whose result nobody consumes still runs, because the call itself created it.

This is where almost everyone gets confused. Slow down: every later concept depends on it.

Airflow schedules intervals, not moments. A `@daily` DAG with `start_date=2024-05-01` produces a run for the interval covering 1 May, and that run starts **at the end of the interval**, just after midnight on 2 May. The data for 1 May is only complete once 1 May is over.



2024-05-01 00:00

Interval starts

`data_interval_start` . Nothing runs yet.



2024-05-02 00:00

Interval ends

`data_interval_end` . The run is now scheduled.



2024-05-02 00:00:05

The run starts

The scheduler creates the DAG run and queues its first tasks.



2024-05-02 00:02

Tasks complete

Logical date is 2024-05-01, the interval rather than the wall clock.

The parameter values, in the order they cause trouble:

Parameter	Means	Common mistake
<code>start_date</code>	The first interval Airflow considers	Setting it to "today" and wondering why nothing ran



<code>schedule</code>	How often. cron, preset, timedelta, or <code>None</code>	Assuming it fires at the start of the interval
<code>catchup</code>	Whether to run every missed interval since <code>start_date</code>	Leaving it <code>True</code> with an old start date
<code>end_date</code>	Stop scheduling after this	Rarely needed; useful for a migration

PYTHON

Copy

```
schedule="@daily"           # midnight, every day
schedule="0 2 * * *"        # 02:00 every day, cron
schedule="0 9 * * 1-5"      # 09:00 on weekdays
schedule=timedelta(hours=6) # every six hours from start_date
schedule=None               # manual and triggered runs only
schedule="@once"            # exactly once
```

`catchup=True` with an old start date will flood your cluster

A DAG with `start_date=datetime(2022, 1, 1)`, `schedule="@daily"`, and `catchup=True` creates roughly **eight hundred** DAG runs the moment you unpause it. Default to `catchup=False` and backfill deliberately when you want history. There is a command for it, covered later.

The other detail that costs people an afternoon: **use the interval, not `datetime.now()`**. A task that queries "yesterday" by calling `now()` produces different results on a rerun than it did originally, which destroys reproducibility. Airflow gives you the interval as a template variable.

PYTHON

Copy

```
BashOperator(
    task_id="extract",
    bash_command="python extract.py --date {{ ds }}",    # 2024-05-01
)
```



<code>{{ ds }}</code>	2024-05-01 , the logical date
<code>{{ ds_nodash }}</code>	20240501
<code>{{ data_interval_start }}</code>	2024-05-01T00:00:00+00:00
<code>{{ data_interval_end }}</code>	2024-05-02T00:00:00+00:00
<code>{{ prev_ds }}</code> / <code>{{ next_ds }}</code>	The neighbouring intervals
<code>{{ dag_run.run_id }}</code>	This run's identifier
<code>{{ ts }}</code>	The full logical timestamp

► TRY IT: THE ONE THAT MAKES SCHEDULING CLICK

1. Create a `@daily` DAG with `start_date` three days ago and `catchup=True` . Unpause it and count the runs.
2. Add a `BashOperator` running `echo "logical={{ ds }} now=$(date -I)"` and compare the two values in each run's log.
3. Set `catchup=False` , clear everything, and confirm only the most recent interval runs.
4. Change `start_date` to tomorrow and confirm nothing runs at all.

Expected result – three runs appear immediately with `catchup` on, and in each log the logical date differs from the wall clock. Seeing `logical=2024-05-01 now=2024-05-04` in a backfilled run is the moment the interval model becomes obvious rather than confusing.

Retries, timeouts, and `default_args`

A pipeline that fails permanently on a transient network blip is not production-ready. Airflow's retry handling is a few parameters, and setting them is the difference between a pipeline you trust and one you babysit.



```
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.bash import BashOperator

default_args = {
    "owner": "data-platform",
    "retries": 3,
    "retry_delay": timedelta(minutes=5),
    "retry_exponential_backoff": True,
    "max_retry_delay": timedelta(minutes=30),
    "execution_timeout": timedelta(minutes=45),
    "email_on_failure": False,
}

with DAG(
    dag_id="robust_pipeline",
    start_date=datetime(2024, 1, 1),
    schedule="@daily",
    catchup=False,
    default_args=default_args,
    dagrun_timeout=timedelta(hours=2),
    max_active_runs=1,
    tags=["production"],
) as dag:

    fetch = BashOperator(
        task_id="fetch_api",
        bash_command="python fetch.py --date {{ ds }}",
        retries=5,                                # overrides the default for this task
    )
```

Parameter	Scope	Does
<code>retries</code>	Task	How many times to retry after a failure
<code>retry_delay</code>	Task	How long to wait before retrying
<code>retry_exponential_backoff</code>	Task	Doubles the delay each attempt
<code>execution_timeout</code>	Task	Kill the task if it exceeds this



<code>dagrun_timeout</code>	DAG	Fail the whole run if it exceeds this
<code>max_active_runs</code>	DAG	How many runs of this DAG may overlap
<code>max_active_tasks</code>	DAG	Task-level concurrency within the DAG
<code>depends_on_past</code>	Task	Only run if the previous interval's instance succeeded

`default_args` is applied to every task in the DAG, and any task can override an individual value. That pattern (sensible defaults at the DAG level, exceptions at the task level) is how real DAGs stay readable.

Retries are only safe if your task is idempotent

Running a task twice must produce the same result as running it once. A task that `INSERT`s rows will duplicate them on retry; a task that `DELETE`s the target partition and then inserts will not. Design for reruns and retries become free. Otherwise every retry is a data-quality incident waiting to happen.

`execution_timeout` deserves special mention because its absence is the classic outage: a task that hangs on a network read holds a worker slot indefinitely, and with `max_active_runs` unset the next intervals pile up behind it until the whole DAG is stuck. One `timedelta` prevents it.

► TRY IT

1. Add a task that fails: `bash_command="exit 1"` , with `retries=2` and `retry_delay=timedelta(seconds=10)` .
2. Watch the UI cycle through `up_for_retry` and read the log. All three attempts are there.
3. Add `execution_timeout=timedelta(seconds=15)` to a task running `sleep 60` and watch it be killed.
4. Now make the failing task succeed on its third attempt using a file-based counter, and confirm the DAG run turns green.

Expected result – a task that retries visibly, one that is killed on timeout, and one that eventually succeeds and marks the run green. That last case is what



Passing data with XCom, and its limits

Tasks are separate processes, often on separate machines. XCom, cross-communication, is how they pass small values.

PYTHON

Copy

```
@task
def get_record_count():
    return 4_213                                # pushed to XCom automatically

@task
def report(count: int):
    print(f"Processed {count} records")

report(get_record_count())
```

With TaskFlow, returning a value pushes it and consuming it pulls it. With classic operators you do it by hand:

PYTHON

Copy

```
def _extract(**context):
    context["ti"].xcom_push(key="row_count", value=4213)

def _report(**context):
    count = context["ti"].xcom_pull(task_ids="extract", key="row_count")
    print(f"Processed {count} records")
```

In a template, `ti.xcom_pull` is available directly:

PYTHON

Copy



```
dash_command= echo rows. {{ ti.xcom_pull(task_id='extract') }} ,  
)
```

XCom is for metadata, not data

Every XCom value is serialised into the **metadata database**. A row count, a file path, a partition name, a model version are all fine. A dataframe, a file's contents, a list of a million ids are not: you will bloat the database, slow every UI page, and eventually hit a size limit. The correct pattern is to write the data to object storage and pass the **path** through XCom.

✓ Pass through XCom

- A row count or a checksum
- An S3 key or a table partition
- A model version or a run id
- A small dict of decisions

✗ Pass through storage instead

- A dataframe of any size
- A file's contents
- Query results
- Anything you would not put in a database cell

► TRY IT

1. Write a TaskFlow DAG where `extract` returns a dict and `load` consumes it. Confirm it works.
2. Open the task instance details in the UI and find the **XCom** tab, then read the stored value.
3. Now return a large object, `list(range(500_000))`, and observe the effect on the run and the XCom page.
4. Rewrite it properly: write the list to a file, return the path, and read the file in the next task.

Expected result – a visible XCom value in the UI, then a painfully slow one, then a clean version that passes a path. Doing the wrong version once makes the "metadata, not data" rule permanent.



Credentials do not belong in your DAG file. Airflow has two stores for configuration, and they have different jobs.

Connections describe how to reach an external system. Create one in **Admin → Connections**, or from the CLI:

BASH

Copy

```
airflow connections add 'warehouse' \  
  --conn-type 'postgres' \  
  --conn-host 'db.internal' \  
  --conn-port 5432 \  
  --conn-schema 'analytics' \  
  --conn-login 'airflow' \  
  --conn-password 'secret'
```

PYTHON

Copy

```
from airflow.providers.postgres.hooks.postgres import PostgresHook  
from airflow.providers.common.sql.operators.sql import SQLExecuteQueryOperator  
  
# In an operator: name the connection, never the credentials  
SQLExecuteQueryOperator(  
    task_id="daily_rollup",  
    conn_id="warehouse",  
    sql="sql/daily_rollup.sql",  
)  
  
# In Python: a hook gives you a real client  
@task  
def count_rows():  
    hook = PostgresHook(postgres_conn_id="warehouse")  
    return hook.get_first("SELECT count(*) FROM events")[0]
```

Variables hold configuration values that are not credentials, like a bucket name, a threshold, or a feature flag:



```
from airflow.models import Variable

bucket = Variable.get("data_bucket")
config = Variable.get("pipeline_config", deserialize_json=True)
retries = Variable.get("max_retries", default_var=3)
```

Store	For	Read in a DAG with
Connection	Host, port, credentials, extras for a system	<code>conn_id=</code> on an operator, or a Hook
Variable	Non-secret configuration values	<code>Variable.get()</code> , or <code>{{ var.value.name }}</code>

Never call `Variable.get()` at the top level of a DAG file

Top-level code runs on **every parse**, every thirty seconds, for every DAG file. A `Variable.get()` up there is a database query on that schedule, and ten such DAGs will visibly degrade your scheduler. Use it inside a task, or use the template form `{{ var.value.data_bucket }}` , which is only resolved at run time.

► TRY IT

1. Create a connection in the UI to any database you have. Even the Airflow metadata Postgres will do for practice.
2. Write a task that uses a Hook to run `SELECT 1` and returns the result.
3. Create a Variable and read it two ways: with `Variable.get()` inside a task, and with `{{ var.value.name }}` in a Bash command.
4. Now put `Variable.get()` at the top of the file, and watch the DAG-parse duration in **Admin** → **DAG Processing**.

Expected result – a working database task with no credentials in your code, and a measurable parse-time increase from one misplaced `Variable.get()` . That measurement is the most convincing argument against top-level code you will find.



That `{{ ds }}` is Jinja. Airflow renders templated fields at run time, which is how a task knows which interval it is working on.

PYTHON

Copy

```
BashOperator(
    task_id="load_partition",
    bash_command=(
        "python load.py "
        "--source s3://{{ var.value.data_bucket }}/raw/{{ ds }}/ "
        "--target events_{{ ds_nodash }}"
        "--run-id {{ dag_run.run_id }}"
    ),
)

SQLExecuteQueryOperator(
    task_id="rollup",
    conn_id="warehouse",
    sql="sql/rollup.sql",          # the FILE is templated too
    params={"lookback_days": 7},
)
```

sql/rollup.sql

Copy

```
DELETE FROM daily_stats WHERE day = '{{ ds }}';

INSERT INTO daily_stats (day, events, users)
SELECT '{{ ds }}', count(*), count(DISTINCT user_id)
FROM events
WHERE event_date BETWEEN '{{ macros.ds_add(ds, -params.lookback_days) }}'
      AND '{{ ds }}';
```

Note the shape of that SQL: a `DELETE` for the target partition followed by an `INSERT`. That is the **idempotent** pattern. Rerun it as many times as you like and the result is identical, which is what makes retries and backfills safe.

Two things to know about templating:



and `parameters`. Putting `{{ ds }}` in an untemplated field gives you the literal string.

`macros` gives you date arithmetic without importing anything: `{{ macros.ds_add(ds, -7) }}`, `{{ macros.datetime.now() }}`, `{{ macros.ds_format(ds, "%Y-%m-%d", "%d/%m/%Y") }}`.

Check the rendered value before debugging your code

A task instance's **Rendered Template** tab in the UI shows what the field became after templating. Half of "my SQL is wrong" turns out to be a template that resolved differently than expected, and this tab answers it in one click.

► TRY IT

1. Write a task whose command includes `{{ ds }}`, `{{ ds_nodash }}`, and `{{ macros.ds_add(ds, -7) }}`.
2. Run it, then open **Rendered Template** and compare against the log output.
3. Put `{{ ds }}` into a field that is

Expected result – not

templated (`task_id`, for instance) and see the literal string appear.

4. Write the idempotent SQL above against a scratch table, run it twice, and confirm the row count does not double.

Expected result – a rendered template you can read before the task runs, a literal `{{ ds }}` in an untemplated field, and SQL that is safe to rerun. That last property is the foundation everything else in Airflow relies on.

Sensors: waiting without wasting a worker

Sometimes a task cannot start until something external exists: a file lands, a table is populated, an upstream team finishes. A **sensor** is an operator that waits.



```
from airflow.providers.amazon.aws.sensors.s3 import S3KeySensor
from airflow.sensors.filesystem import FileSensor

wait_for_file = S3KeySensor(
    task_id="wait_for_extract",
    bucket_name="raw-data",
    bucket_key="events/{{ ds }}/_SUCCESS",
    aws_conn_id="aws_default",
    poke_interval=60,                # check every minute
    timeout=60 * 60 * 4,             # give up after four hours
    mode="reschedule",              # ← the important one
)
```

The parameter that matters most is `mode`, and getting it wrong is one of the classic ways to deadlock a cluster:

Mode	Behaviour	Cost
<code>poke</code> (default)	Holds a worker slot for the entire wait	One slot per waiting sensor
<code>reschedule</code>	Releases the slot between checks	Almost nothing while waiting

Sensors in `poke` mode can deadlock your cluster

Ten sensors each waiting four hours in `poke` mode occupy ten worker slots for four hours. If you have sixteen slots, six real tasks can run and everything else queues, including the upstream tasks the sensors are waiting for. Use `mode="reschedule"` for anything that might wait more than a couple of minutes, and **always set a `timeout`**. Mid level covers deferrable operators, which are better still.

Two more waiting patterns worth knowing now:

PYTHON

Copy

```
from airflow.sensors.external_task import ExternalTaskSensor

# Wait for a task in a different DAG's matching interval
wait_upstream = ExternalTaskSensor(
```



```
mode="reschedule",  
timeout=60 * 60 * 2,  
)
```

PYTHON

Copy

```
from airflow.sensors.python import PythonSensor  
  
@task.sensor(poke_interval=120, timeout=3600, mode="reschedule")  
def wait_for_rows():  
    hook = PostgresHook(postgres_conn_id="warehouse")  
    count = hook.get_first("SELECT count(*) FROM staging WHERE day = %s", ("{{ ds }}",))  
    return count > 0 # truthy return means "done waiting"
```

► TRY IT

1. Write a `FileSensor` waiting for a path that does not exist, with `poke_interval=10` and `timeout=120` .
2. Run it in `poke` mode and watch the task hold a slot in the UI while it waits.
3. Let it time out and read the failure message.
4. Switch to `mode="reschedule"` , rerun, and notice the task alternates between `up_for_reschedule` and running.
5. Create the file mid-wait and watch the sensor succeed.

Expected result – a sensor that occupies a slot in one mode and releases it in the other, and a timeout that fails cleanly rather than waiting forever. That state difference in the UI is the whole argument for `reschedule` .

Branching and trigger rules

Not every run should do the same thing. Airflow's branching operator picks a path at run time.



```
from airflow.operators.python import BranchPythonOperator
from airflow.operators.empty import EmptyOperator
from airflow.utils.trigger_rule import TriggerRule

def _choose_path(**context):
    count = context["ti"].xcom_pull(task_ids="extract", key="return_value")
    return "full_reload" if count > 100_000 else "incremental_load"

branch = BranchPythonOperator(task_id="choose_path", python_callable=_choose_path)

full = EmptyOperator(task_id="full_reload")
incr = EmptyOperator(task_id="incremental_load")

join = EmptyOperator(
    task_id="join",
    trigger_rule=TriggerRule.NONE_FAILED_MIN_ONE_SUCCESS,  # ← required
)

extract >> branch >> [full, incr] >> join
```

The branching callable returns a `task_id`, or a list of them, and Airflow **skips** every other branch. That skipping is why the join task needs a trigger rule: by default a task requires `all` upstreams to have succeeded, and a skipped upstream is not a success.

Trigger rule	Runs when
<code>all_success</code> (default)	Every upstream succeeded
<code>all_done</code>	Every upstream finished, whatever the outcome
<code>one_success</code>	At least one upstream succeeded
<code>one_failed</code>	At least one upstream failed
<code>none_failed</code>	Nothing failed; skips are acceptable
<code>none_failed_min_one_success</code>	Nothing failed and at least one ran: the branch-join rule
<code>all_skipped</code>	Every upstream was skipped



PYTHON

Copy

```
@task.branch
def choose_path(count: int) → str:
    return "full_reload" if count > 100_000 else "incremental_load"
```

`all_done` is the rule for cleanup tasks that must run regardless:

PYTHON

Copy

```
cleanup = BashOperator(
    task_id="cleanup",
    bash_command="rm -rf /tmp/staging/{{ ds }}",
    trigger_rule=TriggerRule.ALL_DONE,      # even if everything upstream failed
)
```

A skipped task is not a failed task, and it is not a success either

Skips propagate downstream. A join task with the default `all_success` rule after a branch will itself be skipped, silently, and the rest of your DAG with it. If a branch is followed by anything, the join needs an explicit trigger rule. This is the single most common branching bug.

► TRY IT

1. Build the branch above with a callable that picks based on a Variable you can change.
2. Run it and confirm one branch is green and the other is pink (skipped).
3. Remove the `trigger_rule` from the join and rerun, then watch the join get skipped too.
4. Add a cleanup task with `ALL_DONE` and make an upstream task fail. Confirm cleanup still runs.

Expected result – a working branch, then the silent skip cascade when the trigger rule is missing, then a cleanup task that survives a failure. Steps three and four together cover most of what trigger rules are for.



The interface is your primary debugging tool, and knowing which view answers which question saves real time.

View	Answers
Grid	"What happened across the last N runs?" The default, and the one to live in
Graph	"What is the dependency structure, and where did it stop?"
Gantt	"Which task is the bottleneck?" Durations as a timeline
Calendar	"Which days failed?" A heatmap over months
Code	"What is the scheduler running?" The parsed source
Task Instance → Logs	"Why did it fail?"
Task Instance → Rendered Template	"What did my template become?"
Task Instance → XCom	"What did it pass downstream?"
Admin → DAG Processing	"Why is the scheduler slow?" Parse times per file

The colours you need to recognise:

State	Means
Green <code>success</code>	Done
Red <code>failed</code>	Failed after exhausting retries
Yellow <code>up_for_retry</code>	Failed, will try again
Orange <code>up_for_reschedule</code>	A sensor waiting in reschedule mode
Light green <code>running</code>	Executing now



Grey <code>queued</code>	waiting for a worker slot
Pink <code>skipped</code>	Skipped by a branch or a trigger rule
Dark red <code>upstream_failed</code>	Never ran; an upstream failed
White <code>none</code>	Not yet scheduled

The four actions on a task instance that you will use constantly:

Action	Does
Clear	Wipe the state so the scheduler reruns it: the main way to retry
Mark Success	Force it green without running it: use sparingly, and never to hide a problem
Mark Failed	Force it red, stopping downstream work
Clear + Downstream	Rerun this task and everything after it

"Clear" is the verb, not "rerun"

There is no rerun button. You **clear** a task instance's state, and the scheduler notices there is now a task instance with no state and runs it. Understanding that indirection explains why clearing works on old runs, why it can cascade downstream, and why the scheduler must be healthy for it to take effect.

► TRY IT

1. Make a DAG with one task that fails and one that sleeps for thirty seconds. Let it run.
2. Find every state colour you can: success, failed, upstream_failed, skipped, queued.
3. Open the **Gantt** view and identify the longest task.
4. Clear the failed task and watch it rerun. Then use **Clear + Downstream** on an earlier task.
5. Open **Admin** → **DAG Processing** and note your DAG's parse duration.

Expected result – a tour of every state you will meet, a Gantt chart naming your bottleneck, and a clear-and-rerun you performed yourself. The parse-duration



The CLI and backfills: running history on purpose

The UI is convenient; the CLI is what you use when you need to be precise, and it is how you run a backfill.

BASH

Copy

```
# Inspect
airflow dags list
airflow dags list-runs -d my_dag
airflow tasks list my_dag --tree

# Validate before deploying
python dags/my_dag.py # does the file even import?
airflow dags list-import-errors

# Test a single task without touching the database
airflow tasks test my_dag my_task 2024-05-01

# Test a whole DAG run locally
airflow dags test my_dag 2024-05-01

# Trigger and manage
airflow dags trigger my_dag
airflow dags trigger my_dag --conf '{"reprocess": true}'
airflow dags pause my_dag
airflow dags unpause my_dag

# Backfill a date range
airflow dags backfill my_dag \
    --start-date 2024-04-01 \
    --end-date 2024-04-30

# Clear a range of task instances
airflow tasks clear my_dag \
    --start-date 2024-04-01 --end-date 2024-04-30 \
    --task-regex "transform.*" --downstream
```



your terminal, and it does **not** record state in the metadata database. It is the fastest possible feedback loop for debugging a task, and it works on any date.

`airflow dags backfill` creates runs for a historical range. It is how you populate a new DAG's history, or reprocess after fixing a bug. It only works properly if your tasks are idempotent, which is why that property keeps coming up.

A backfill can overwhelm everything else

Thirty days of a DAG with twenty tasks is six hundred task instances competing with your live pipelines for worker slots. Set `max_active_runs` on the DAG, and consider `--reset-dagruns` semantics carefully. Backfilling a month at full concurrency on a shared cluster is a good way to delay every other team's pipeline.

► TRY IT

1. Run `airflow tasks test my_dag my_task 2024-05-01` and watch the log stream to your terminal.
2. Confirm the UI shows no new task instance for that run. The test left no state.
3. Break the task's code and run `python dags/my_dag.py` to see the import error before deploying.
4. Set `max_active_runs=1`, then backfill a five-day range and watch the runs execute one at a time.

Expected result – an instant feedback loop that leaves no trace, an import error caught before the scheduler saw it, and a rate-limited backfill. That first one will change your development habits more than anything else on this page.

Top-level code, and the rule that matters most

This is the single most important performance rule in Airflow, and it follows directly from the architecture: **the scheduler executes every DAG file, repeatedly, just to find the DAG objects.**



- API calls
- `Variable.get()`
- Reading files
- Heavy imports (pandas, torch)

- Only: cheap constants
- Only: light imports
- Every 30 seconds, per file, forever
- Slow code here slows the whole scheduler

PYTHON

Copy

```
# WRONG – this runs on every single parse
import pandas as pd
config = requests.get("https://config.internal/pipeline").json()
tables = PostgresHook("warehouse").get_records("SELECT name FROM tables")

with DAG(...) as dag:
    for table in tables:
        ...

# RIGHT – the expensive work happens at run time
with DAG(...) as dag:

    @task
    def get_tables():
        import pandas as pd                # imported inside the task
        return PostgresHook("warehouse").get_records("SELECT name FROM tables")

    @task
    def process(table: str):
        ...

    process.expand(table=get_tables())      # dynamic mapping, Mid level
```

The cost is easy to underestimate. A `Variable.get()` at the top level of one DAG is one query every thirty seconds, 2,880 a day. Across twenty DAG files that is 57,600 queries a day for values nobody read. An API call up there adds its latency to every parse cycle.

BASH

Copy

```
# Measure it
airflow dags list-import-errors
```



The **Admin → DAG Processing** page shows the same thing in the UI: the name, number of DAGs, and parse duration. Anything over a second deserves a look; anything over five needs fixing.

The one-line test

Ask of every line outside a task: **"do I want this to run every thirty seconds forever?"** If the answer is no, it belongs inside a task. That question resolves every top-level-code question you will have.

► TRY IT

1. Note your DAG's parse duration in **Admin → DAG Processing**.
2. Add `import time; time.sleep(3)` at the top level and watch the duration change.
3. Add `Variable.get("something")` at the top level and observe the effect.
4. Move both inside a task and confirm the parse duration returns to normal.
5. Run `airflow dags report` and compare the numbers against the UI.

Expected result – a parse duration you made worse on purpose and then fixed. Watching that number move in response to your own change is what makes the rule stick, and it is the measurement you will reach for whenever Airflow feels slow.

Putting it all together

Everything above in one DAG. Nothing here is new. Read it as a whole and you should be able to justify every line.

dags/daily_events_pipeline.py

Copy

```
"""Daily events pipeline: land → validate → transform → publish → notify."""  
from datetime import datetime, timedelta
```



```
from airflow.providers.common.sql.operators.sql import SQLExecuteQueryOperator
from airflow.providers.postgres.hooks.postgres import PostgresHook
from airflow.sensors.filesystem import FileSensor
from airflow.utils.trigger_rule import TriggerRule

# Sensible defaults for every task; individual tasks override as needed.
default_args = {
    "owner": "data-platform",
    "retries": 3,
    "retry_delay": timedelta(minutes=5),
    "retry_exponential_backoff": True,
    "execution_timeout": timedelta(minutes=30),
}

@dag(
    dag_id="daily_events_pipeline",
    start_date=datetime(2024, 1, 1),
    schedule="0 2 * * *",          # 02:00, after the source system settles
    catchup=False,                 # backfill deliberately, never accidentally
    default_args=default_args,
    dagrun_timeout=timedelta(hours=3), # the whole run has a ceiling
    max_active_runs=1,             # no overlapping runs on the same table
    tags=["production", "events"],
    doc_md=__doc__,                # this docstring renders in the UI
)
def daily_events_pipeline():

    start = EmptyOperator(task_id="start")

    # reschedule mode: releases the worker slot between checks.
    # timeout: never wait forever.
    wait_for_extract = FileSensor(
        task_id="wait_for_extract",
        filepath="/data/landing/events_{{ ds_nodash }}.parquet",
        poke_interval=120,
        timeout=60 * 60 * 2,
        mode="reschedule",
    )

    @task
    def validate() → int:
        """Row count is returned through XCom – a number, not the data."""
        hook = PostgresHook(postgres_conn_id="warehouse") # a connection, not a password
        count = hook.get_first()
```



```
    if count == 0:
        raise ValueError("No rows landed for {{ ds }}")
    return count

# Idempotent by construction: DELETE the partition, then INSERT it.
transform = SQLExecuteQueryOperator(
    task_id="transform",
    conn_id="warehouse",
    sql="sql/daily_rollup.sql",      # templated file, kept out of the DAG
    params={"lookback_days": 7},
)

@task
def publish(row_count: int) → str:
    """Returns a path, not a payload."""
    target = f"s3://analytics/daily/{{{{ ds }}}}/events.parquet"
    print(f"Published {row_count} rows to {target}")
    return target

notify = BashOperator(
    task_id="notify",
    bash_command='echo "events {{ ds }} done: {{ ti.xcom_pull(task_ids=\'publish\')'"
)

# ALL_DONE: cleanup must happen whether the run succeeded or not.
cleanup = BashOperator(
    task_id="cleanup",
    bash_command="rm -f /tmp/events_{{ ds_nodash }}.tmp",
    trigger_rule=TriggerRule.ALL_DONE,
)

end = EmptyOperator(task_id="end")

row_count = validate()
start >> wait_for_extract >> row_count >> transform
transform >> publish(row_count) >> notify >> cleanup >> end

daily_events_pipeline()
```



MLOps MENA
COMMUNITY

العربية



Join the Community ↗





1. Adapt this DAG to a real workflow of yours, even a small one, and get it green.
2. Verify the important properties actively: run the transform twice and confirm the row count does not double; make the sensor time out and confirm it fails cleanly; make a task fail and confirm `cleanup` still runs.
3. Backfill three historical days with `max_active_runs=1` and confirm each produces correct, non-duplicated output.
4. Check **Admin** → **DAG Processing** and confirm the parse duration is well under a second.

Expected result – a pipeline that is safe to retry, safe to backfill, cleans up after itself, and parses fast. Those four properties are what separate a DAG that works from one you can leave running unattended, and you will learn more building it than from a second pass over the page.

What you can now do, and what comes next

You can write a DAG in both styles, reason correctly about intervals and logical dates, set retries and timeouts that make a pipeline trustworthy, pass metadata between tasks without abusing the database, keep credentials out of your code, template dates and SQL, wait on external systems without deadlocking your workers, branch conditionally and handle the trigger rules that come with it, read every view in the UI, use the CLI for fast feedback and deliberate backfills, and keep the scheduler fast by understanding top-level code. That is a working practitioner's toolkit, enough to own real DAGs on a shared Airflow.

Can you...	
Name the five core components?	Scheduler, executor, worker, metadata DB, webserver
Say what Airflow is <code>not</code> for?	Processing data itself. It orchestrates
Explain why a <code>@daily</code> run starts after midnight?	It schedules intervals, and the interval must end



Say what <code>catchup=true</code> does with an old start date?	Creates every missed run at once
Give the reason to use <code>{{ ds }}</code> over <code>now()</code> ?	Reruns and backfills must be reproducible
Explain why XCom is not for dataframes?	It is serialised into the metadata database
Say where credentials belong?	A Connection, referenced by <code>conn_id</code>
Name the sensor mode that frees a worker slot?	<code>reschedule</code>
Say why a join after a branch needs a trigger rule?	Skips are neither success nor failure
Explain what "clear" does?	Wipes state so the scheduler reruns it
Give the top-level code rule?	Would you want this every thirty seconds?
Say what makes a task safe to retry?	Idempotence: delete the partition, then write

Mid-level takes every one of those topics further: the executors and what each one costs, dynamic task mapping with `.expand()` , deferrable operators that replace sensors entirely, TaskGroups and DAG factories, datasets and data-aware scheduling, pools and priority weights for concurrency control, SLAs and callbacks, custom operators and hooks, testing DAGs properly, and the CI patterns that stop a broken DAG reaching production.





MLOps MENA
COMMUNITY

Free MLOps and AI learning for engineers across the Middle East and North Africa — live sessions, three structured roadmaps, open cohort courses, study groups, and mentorship. We also train, staff, and build for companies across the region.



LEARN

[Roadmaps](#)

[Student Guides](#)

[Courses](#)

[The MLOps Practitioner](#)

[Sessions](#)

[Articles](#)

COMMUNITY

[Meet the team](#)

[Services for companies](#)

[FAQ](#)

[Contact us](#)

[Privacy Policy](#)

WHERE WE ARE

[WhatsApp](#)



MLOps MENA
COMMUNITY

العربية



Join the Community ↗



X

YouTube

GitHub

Zomra



Brainsmingle space

Free to join this week with code **FREECORETEAM**

© 2026 MLOps MENA Community. Free learning for the community.

Built by the community, for the community.